Lecture L01

*Blind review — please score this lecture on row **L01** in the Excel form (1–10 per dimension).*

Normalization Forms and Database Schema Design (1NF–BCNF)

By the end of this lecture, students will be able to:

1. Define and explain the concept of normalization in database schema design.
2. Describe the fundamental principles of the First, Second, and Third Normal Forms (1NF, 2NF, and 3NF).
3. Identify the characteristics of a relation in Boyce-Codd Normal Form (BCNF).
4. Distinguish between denormalization and normalization.
5. Apply the concepts of normalization to a real-world example, specifically the TEACH relation.

Imagine a large online retailer, such as Amazon, with a massive relation containing customer information, including their ID, name, and contact details. However, this relation is prone to data redundancy and inconsistencies. To address this issue, normalization is employed to break down the relation into smaller, more manageable components, each with its own set of attributes and constraints.

Worked Example: 1NF

Consider a university's student information system, where a single relation contains student demographics, academic records, and extracurricular activities. To achieve 1NF, we would decompose this relation into smaller components, each with its own set of attributes. For instance, we could create separate relations for student demographics, course enrollment, and academic performance. This decomposition ensures that each attribute can have only one value, and it cannot be a relation itself. 1NF is a necessary step towards achieving higher levels of normalization, but it is not sufficient to prevent data redundancy.

Worked Example: 2NF

Suppose a popular e-commerce platform, such as eBay, has a relation containing auction information, including the auction ID, item ID, and bidding history. To achieve 2NF, we would decompose this relation into smaller components, each with its own set of attributes. For instance, we could create a separate relation for auction details, which would eliminate the need for storing redundant auction information.

Worked Example: 3NF

Imagine a relation for a social media platform, such as Facebook, containing user information, including their ID, name, and friend list. To achieve 3NF, we would decompose this relation into smaller components, each with its own set of attributes. For instance, we could create separate relations for user profiles, friend connections, and post history, which would eliminate the need for storing redundant data.

Worked Example: BCNF

Consider a relation for a large healthcare organization, containing patient information, including their ID, name, and medical history. To achieve BCNF, we would decompose this relation into smaller components, each with its own set of attributes. For instance, we could create a separate relation for patient demographics, which would eliminate the need for storing redundant information.

Worked Example: Denormalization vs. Normalization

Suppose a travel booking platform, such as Expedia, has a relation containing booking information, including the booking ID, customer ID, and travel itinerary. To denormalize this relation, we would store redundant booking information, such as the travel dates and destinations, for each customer. However,

this approach would lead to data redundancy and inconsistencies. On the other hand, normalizing this relation would break it down into smaller components, each with its own set of attributes, eliminating the need for storing redundant data.

Try This Yourself

1. Identify the attributes in a given relation that can be decomposed into smaller components to achieve 1NF.
2. Determine the non-key attributes in a relation that are dependent on the entire primary key to achieve 2NF.
3. Identify the transitive dependencies in a relation that can be eliminated to achieve 3NF.
4. Apply the concepts of normalization to a real-world example, specifically the TEACH relation.

Normalizing a database is like solving a complex puzzle, where each piece of data is a crucial part of the overall picture. By carefully decomposing the data into smaller components, we can create a stable structure that minimizes data redundancy and improves data integrity, ensuring that our database is robust, efficient, and easy to maintain.